# Anomaly Detection and Isolation of Compromised Sensors and Actuators in Water Distribution Networks

*A Deep Learning Approach Using GRU Models and Signature Vectors on the BATADAL Dataset*

*Course: 42.705: Data Science Design Project*

*Singapore University of Technology and Design (SUTD)*


**Supervisor / Instructor:**

Prof. Carlos Gerardo Murguia Rendon

Associate Professor - ESD Pillar

**Team Members**

Fu Ruiqi **1009646**

Nithishna **1009665**

Ezhilmurugan Udayakumar **1009666**

Liang peirong **1009669**

## Abstract

*Water distribution networks (WDNs) are critical infrastructures whose safe and reliable operation can be disrupted by cyber-physical attacks or unexpected faults. Detecting these anomalies in real time is challenging due to the nonlinear dynamics, temporal dependencies, and noisy nature of sensor data. In this work, we propose a residual-based anomaly detection and fault isolation framework built on Gated Recurrent Unit (GRU) models. Each GRU learns the relational behaviour of an individual component, and deviations between predicted and observed values are used as residuals that can be used by the computed threshold for anomaly detection. To strengthen robustness and evaluate generalisation, we introduce a virtual attack injection mechanism that perturbs clean data with synthetic signals, generating residual patterns and binary signature vectors. These virtual signatures are then compared with real attack signatures from the BATADAL benchmark dataset to assess the similarity of the affected components.*

*The experimental study demonstrates that the approach can identify anomalous events across tanks, pumps, and valves, while producing residuals that are consistent with both real and synthetic attack conditions. Signature analysis further confirms the method's ability to provide interpretable indicators of frequently compromised components. By combining deep learning prediction accuracy, residual-based interpretability, and robustness validation through synthetic stress testing, the framework offers a comprehensive solution for anomaly detection and fault isolation. Its modular design also makes it suitable to behave as a digital twin that can be deployed, enabling real-time monitoring and proactive security management of WDNs.*

*Keywords: Water distribution networks, anomaly detection, cyber-physical security, residual analysis, GRU, virtual attack simulation, signature vectors.*

| Chapter | Title | Page |
|---|---|---|
| | Abstract | ii |
| | Executive Summary | vii |
| 1 | Introduction | 1 |
| 1.1 | Background and Motivation | 1 |
| 1.2 | The BATADAL Benchmark | 1 |
| 1.3 | Challenges in Attack Detection | 2 |
| 1.4 | Objectives of This Work | 2 |
| 2 | Methodology and Analysis | 3 |
| 2.1 | Overview of Approach | 3 |
| 2.2 | Data Preprocessing | 4 |
| 2.3 | GRU Modelling | 5 |
| 2.3.1 | Modelling with GRU Networks | 6 |
| 2.4 | Residual Computation and Anomaly Detection | 6 |
| 2.5 | ROC Evaluation and Detection Metrics | 8 |
| 2.6 | Virtual Attack Injection | 8 |
| 2.7 | Signature Vector Construction | 9 |
| 2.8 | Cosine Similarity Matching Signature Vector | 11 |
| 2.9 | Cosine Similarity Matching in Average Residual | 12 |
| 2.10 | Performance | 13 |

| | | |
|---|---|---|
| 3 | Results and Discussion | 14 |
| 3.1 | GRU Model Training and Performance | 14 |
| 3.2 | Residual Analysis and Thresholds | 15 |
| 3.3 | ROC Curve Evaluation | 16 |
| 3.4 | Virtual Attack Injection Result | 18 |
| 3.5 | Signature Vector Analysis | 19 |
| 3.6 | Cosine Similarity Matching Signature Vector | 20 |
| 3.7 | Cosine Similarity Matching in Average Residual | 20 |
| 3.8 | Overall Performance | 21 |
| 4 | Conclusion | 23 |
| 5 | Technical Documentation | 24 |
| 5.1 | Scripts | 24 |
| 5.2 | Folder Structure | 24 |
| 5.3 | Workflow | 25 |
| 6 | Data and Code Links | 26 |
| 6.1 | Datasets Used | 26 |
| 7 | References | 27 |
| 8 | Appendix | 28 |
| 8.1 | Feature–Target Configurations | 30 |
| 8.2 | ROC Curves (Per Component) | 31 |

8.3 Residual Histograms 32

8.4 Cosine Similarity Matrices 33

| Figure | Title | Page |
|---|---|---|
| 1.1 | C-Town Water Distribution Network (from BATADAL dataset, Taormina et al., 2018) | vii |
| 2.4 | L_T1 Residuals | 7 |
| 3.1.1 | Overall Training of the GRU Model | 14 |
| 3.1.2 | Training of Actual vs Predicted | 15 |
| 3.2 | Residual plot | 16 |
| 3.3 | ROC Curve Evaluation | 18 |
| 3.4.1 | Virtual Attack Injection on L_T1 | 18 |
| 3.4.2 | Attack-Injected Signature Vector for L_T1 | 19 |
| 3.5.1 | Attack-Injected Signature Vector for L_T2 | 19 |
| 3.5.2 | Cosign Similarity of Signature vectors | 20 |
| 3.6 | Cosine Similarity Average residuals | 21 |

## Executive Summary

Water distribution networks (WDNs) form the backbone of urban infrastructure, ensuring reliable water supply to communities and industries. However, these systems are increasingly vulnerable to both **cyber intrusions** and **operational faults**, which may compromise service quality, safety, and resilience. The **Battle of the Attack Detection Algorithms (BATADAL)** benchmark provides a realistic dataset that simulates attacks on a medium-sized water distribution network, creating a platform for developing anomaly detection and fault isolation methods.**[1]**

This project addresses the BATADAL challenge by designing an **end-to-end anomaly detection and isolation framework** that combines deep learning accuracy, residual-based interpretability **and robustness validation** through synthetic attack simulation.**[8]**

### Problem Context

Traditional detection methods, such as fixed thresholds or hydraulic model simulations, are limited in their ability to handle the nonlinear, temporal, and noisy nature of WDN data. Furthermore, many existing data-driven methods focus solely on anomaly detection without offering insight into *which component* is under attack, limiting their practical value for operators.

### Description of the C-Town Network

As shown in **Figure 1**, the C-Town network is a benchmark water distribution system used in the BATADAL competition. It consists of **seven storage tanks (T1–T7)**, **eleven pumping stations (PU1–PU11)**, and **two major valves (V1, V2)**, all interconnected by hundreds of pipes and junction nodes. The network is operated by **nine PLCs** that locally control pumps and valves, while a **SCADA layer** provides centralized monitoring. The **reservoir** at the lower right supplies water into the system through pumps PU1–PU3, while other pumps and valves regulate flows across different pressure zones.

This schematic is important because it highlights the interconnected and nonlinear nature of WDNs: changes in one area (e.g., closing V2 or altering PU4 flow) can propagate across the system, affecting tank levels and pressures elsewhere. These dynamics make anomaly detection challenging, since both natural fluctuations (e.g., demand peaks) and malicious attacks can create similar disturbances.

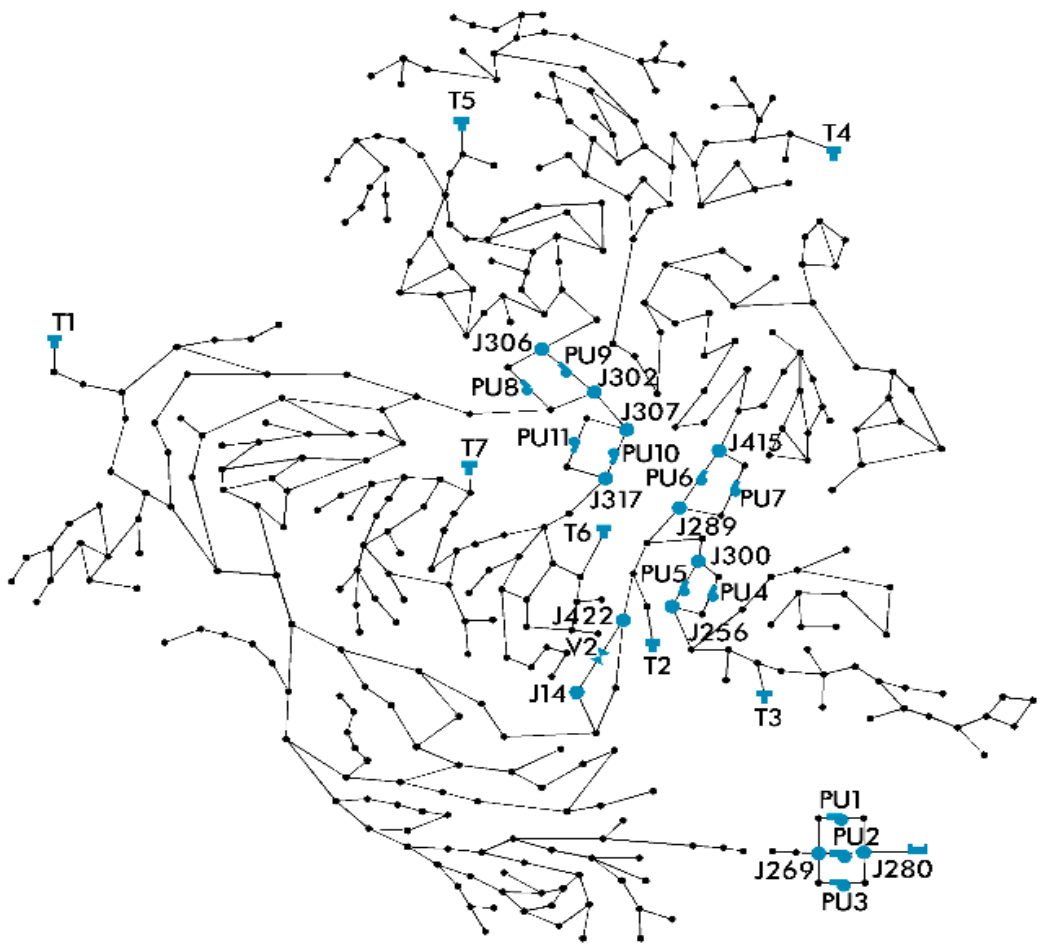


Fig 1.1: The network of C-Town

**Proposed Framework**

The proposed framework unifies deep learning–based anomaly detection, virtual attack stress testing, and signature vector similarity into a single pipeline for fault detection and localisation in Water Distribution Networks (WDNs).**[1]** Unlike traditional methods that only detect anomalies, it links test-time anomalies with synthetically generated virtual attacks, enhancing explainability, robustness, and fault isolation.**[8]**

The framework consists of six layers:

**1. GRU Modelling**

- **Objective:** Model each component's values to compute residuals.

**1.1 Model Input:**

For each target component y(t), we prepare an input sequence consisting of:

$$X(t) = \left[y(t-1), \ldots, y(t-N),\ u(t-1), \ldots, u(t-N),\ u(t)\right]$$

Where:

- y(t−1) … y(t−N): past sensor readings of the target.
- u(t−1) … u(t−N): past actuator values (e.g., pump/valve states, flows, pressures).
- u(t): the current actuator state.

This structure allows the GRU to learn the nonlinear relationship between past sensor values and actuator controls.

**1.2 Model Prediction:**

The GRU model is trained to output the expected value of the target component at time t:

$$\hat{y}(t) = \mathrm{GRU}(X(t))$$

**1.3 Residual Computation:**

Once trained, the model compares the actual vs. predicted value:

$$r(t) = y(t) - \hat{y}(t)$$

Where:

- y(t): actual measured value at time t (e.g., tank level LT1 or pump flow FPU2 ).
- y^(t): predicted value from the GRU model.
- r(t): residual.

If the system is normal, residuals remain small (within thresholds). Under abnormal or attacked conditions, residuals increase significantly.

**2. Threshold Construction for Residual-Based Detection**

**2.1 Residuals on Clean (Attack-Free) Data:**

- After training, each GRU model is evaluated on attack-free data.
- Residuals are computed:

$$r(t) = y(t) - \hat{y}(t)$$

**2.2 Threshold Construction:**

- For each component, thresholds are computed using the **95th percentile** of absolute residuals:

$$\theta = \text{Percentile}_p\big(|r(t)| \ \text{ from clean data}\big)$$

**2.3 Detection Rule:**

- If |r(t)| > θ, the component is flagged as anomalous at time ttt.
- Each component has its threshold to reflect its natural variability.

**2.4 Advantages:**

- Data-driven: Thresholds are derived from healthy operation data.
- Component-specific: Thresholds adapt to each component's dynamics.

**3. Virtual Attack Injection**

- Synthetic perturbations are injected into clean datasets.
- Models are then used to generate predictions under virtual attack conditions.

**4. Average Residual Matching & Cosine Similarity Matching**

- Average residual patterns are computed for both test-time data and virtual attack datasets.
- Cosine similarity is used to measure alignment between the two residual vectors:

$$\text{CosSim}(S_{test}, S_{virt}) = \frac{S_{test} \cdot S_{virt}}{\|S_{test}\| \ \|S_{virt}\|}$$

**5. Anomaly & Signature Analysis**

- Anomalies are detected when residuals cross thresholds.
- Binary signature vectors are constructed to identify which components were most frequently flagged.
- Signature analysis highlights the most likely targeted components.

**6. Performance Evaluation**

- The framework is evaluated by comparing predicted signature vectors against ground truth attack signatures.
- **Cosine similarity** is the main metric, reflecting how well predicted anomalies align with true attacked components.

**Key Outcomes**

- The GRU-based residual framework reliably identified abnormal patterns across tanks, pumps, and valves.
- Virtual attack residuals closely mirrored those observed in real test scenarios, validating the robustness of the approach.**[8]**
- Signature vector analysis confirmed that the framework can not only detect anomalies but also isolate which components were affected.

**Contributions**

The project contributes:

- A reproducible, modular pipeline for anomaly detection in WDNs.
- Integration of predictive deep learning with interpretable residual analysis.
- A novel validation mechanism using virtual attack simulation.

# Chapter 1 – Introduction

## 1. Introduction

### 1.1 Background and Motivation

Water distribution networks (WDNs) are critical infrastructures that ensure continuous access to safe water for urban populations. Modern WDNs are not only physical hydraulic systems of tanks, pumps, pipes, and valves, but also cyber–physical systems (CPSs) that integrate sensing, actuation, and automated control. In typical configurations, sensors measure key state variables (e.g., tank levels, flows, pressures), while actuators (e.g., pumps, valves) regulate system dynamics. These field devices are grouped under the control of programmable logic controllers (PLCs), which execute local control logic and communicate with a centralized Supervisory Control and Data Acquisition (SCADA) system.

While this integration improves efficiency and reliability, it also introduces vulnerabilities. Malicious actors may:

- Spoof or tamper with sensor measurements,
- Override or block actuator commands,
- Manipulate PLC control logic, or
- Disrupt SCADA communication channels.

The consequences can range from minor service disruptions to severe equipment damage and public safety risks.

### 1.2 The BATADAL Benchmark

To address the need for standardized evaluation of detection algorithms, the **Battle of the Attack Detection Algorithms (BATADAL)** competition was launched. BATADAL provides benchmark datasets generated from the **C-Town network**, a medium-sized urban WDN that has become a reference in the research community.**[1]**

As shown in **Figure 1 (C-Town schematic)**, the system consists of:

- **Seven storage tanks** connected through multiple junctions,
- **Eleven pumping stations** and **control valves**,
- **31 sensors** measuring tank levels, pressures, and flows,

- **12 actuators** (pumps and valves),
- Controlled by **nine PLCs** under a SCADA layer.

BATADAL datasets include both normal operation data and periods where **cyber–physical attacks** were simulated, providing ground-truth intervals of anomalies for training and validation.**[5]** A separate **test dataset** is provided without labels, encouraging blind evaluation of detection methods.

## 1.3 Challenges in Attack Detection

Several challenges make anomaly detection in WDNs nontrivial:

1. **Nonlinearity of dynamics** — Hydraulic interactions among tanks, pumps, and valves are nonlinear.
2. **Temporal dependencies** — Current system states depend strongly on past states, requiring sequential modelling.
3. **Operational variability** — Natural fluctuations may resemble attack-induced anomalies.
4. **Unseen attacks** — Algorithms must generalize to novel attacks not seen in training.
5. **Interpretability** — Operators must know *which component* is affected, not only that "an anomaly exists."

## 1.4 Objectives of This Work

This project addresses these challenges by developing a **deep learning–based anomaly detection and isolation framework**, using Gated Recurrent Units (GRUs), for the BATADAL water distribution network. The framework has the following objectives:

- Detect anomalies via **residual thresholding**, ensuring accuracy across heterogeneous components.
- Strengthen weak-performing models using **bidirectional GRUs with Huber loss**.
- Test robustness by introducing **virtual attacks** (sinusoidal perturbations) on clean sensor signals.
- Encode detection results as **binary signature vectors**, enabling component-level fault isolation.
- Quantify similarity between real and virtual attack signatures using **cosine similarity**, verifying generalisation.

# Chapter 2 – Methodology and Analysis

## 2. Methodology and Analysis

## 2.1 Overview of the Framework

The proposed anomaly detection and isolation framework consists of four major stages:

1. **GRU Modelling**: Each component is modelled using a Gated Recurrent Unit (GRU) network trained to learn its relational dynamics.**[5]** By predicting the expected value, deviations (residuals) from the actual measurement can be identified.

   Each component is modelled using a Gated Recurrent Unit (GRU) network trained to capture the nonlinear dynamics between past measurements and control actions.

   Let:

   - $y(t) \in \mathbb{R}^m$: vector of measured component values (e.g., tank levels, flows, pressures) at time t.
   - $u(t) \in \mathbb{R}^n$: vector of actuator states (e.g., pump/valve operations) at time t.
   - N: the lookback window length.
   - The GRU model learns a nonlinear mapping:

     $$\hat{y}(t) = f_{\text{GRU}}\Big(y(t-1), \ldots, y(t-N),\ u(t), u(t-1), \ldots, u(t-N)\Big)$$

     where y^(t) is the predicted value of the component at time t.

2. **Residual-Based Anomaly Detection**: Once the GRU produces a prediction, we compute the residual as the difference between the **true measured value** and the **predicted value**.

   - For each component:

     $$r(t) = y(t) - \hat{y}(t)$$

     Where:

     - y(t): actual measured value at time t (e.g., tank level LT1 or pump flow FPU2 ).
     - y^(t): predicted value from the GRU model.
     - r(t): residual.

- If the absolute residual exceeds a component-specific threshold θ:

$$|r(t)| > \theta \quad \Rightarrow \quad \text{Anomaly detected}$$

- These thresholds are derived from residuals on attack-free data.

3. **Virtual Attack Injection & Prediction**: Synthetic perturbations are introduced into clean datasets to generate *virtual attacks*. Saved models are used to predict the residuals in the virtual attacks and the test dataset from BATADAL.
4. **Anomaly Detection & Signature Vector Matching:** The residuals from the test dataset and virtual attacks are then compared with a saved derived threshold to compute the signature vectors to detect whether it is an attack or benign. A comparison between these signature vectors is also performed using cosine similarity. This provides the information on which component is being attacked most frequently.
5. **Cosine Similarity**: To identify the components which is highly affected by an attack, by measuring the cosine similarity of the residual signature vector from test data with that of a virtual attack and also measuring the threshold-based signature vector from test data with that of a virtual attack.
6. **Performance:** The comparison between the resulting cosine similarity of the anomaly detection and the ground truth labels of the test dataset is also performed using cosine similarity. This provides the overall performance of the model in detecting anomalies.

This step-by-step process ensures both **accurate anomaly detection** and **interpretability at the component level**.

## 2.2 Dataset Description and Preprocessing

The datasets used are provided by the **BATADAL competition**, which simulates cyber-physical attacks on a water distribution network (WDN). Three main datasets were utilised:

- **Dataset 03**: Normal + labelled attacks (training).
- **Dataset 04**: Extended dataset (retraining).
- **Test Dataset**: Evaluation (attacks unlabelled).

The raw datasets contain **43 variables** (7 tanks, 11 pumps, 2 valves, multiple pressures, and actuator statuses).

We use the **BATADAL benchmark datasets**, which consist of time series from the C-Town water distribution network.

- Components monitored:
  - 31 sensors: tank levels (L_Tx), junction pressures (P_Jxxx), and pipe flows (F_PUx).
  - 12 actuators: pumps and valves (ON/OFF or OPEN/CLOSED states).
- Time resolution: 1 hour.
- Datasets:
  - Datasets 03 and 04: include labelled attack intervals, used for training and threshold calibration.
  - Test dataset: unlabelled, used for blind evaluation.

Preprocessing steps:

1. **Scaling:** All variables are normalised using Min–Max scaling to [0, 1]:

$$x' = \frac{x - x_{\text{min}}}{x_{\text{max}} - x_{\text{min}}}$$

2. **Windowing:** Each sample is represented as a sequence of past measurements of length W (e.g., 20 steps):

$$X_t = [y(t-1), y(t-2), \ldots, y(t-W), u(t), u(t-1), \ldots, u(t-W)]$$

   where **y** is the target sensor and **u** represents the actuator states.
3. **Splitting:** Data is split into training (normal), validation, and test subsets. Attack intervals in the training sets are excluded from the training process.

## 2.3 GRU-Based Modelling Models

The GRU model learns normal system behaviour by using past component values & past actuator values, current actuator state to predict the next value of a component.**[6]**

- **Architecture**:
  - GRU layers (64–128 hidden units)
  - Dense output layer with linear or sigmoid activation

- **Loss Function**:
  - Mean Squared Error (MSE) or Huber loss, depending on training stability.
- **Training Setup**:
  - Optimiser: Adam (learning rate = 0.001) **[7]**
  - Batch size: 64
  - Epochs: 50 with early stopping
  - Validation split: 20% of training data

### 2.3.1 Modelling with GRU Networks

- Each physical sensor or actuator (e.g., tank levels, pump flows, valve states, pressure sensors) is modelled individually using a NARX-style Gated Recurrent Unit (GRU) neural network.
- Input: historical sequences of sensor states and current actuator signals, historical sequences of actuator states.
- Output: prediction of the current component value.
- Training is performed on healthy operating data (Dataset03) to learn the nominal behaviour.**[5]**

Let:

- $y(t) \in \mathbb{R}^{m}$ : vector of measured component values (e.g., tank levels, flows, pressures) at time t.
- $u(t) \in \mathbb{R}^{n}$ : vector of actuator states (e.g., pump/valve operations) at time t.
- N: the lookback window length.
- The GRU model learns a nonlinear mapping:

$$\hat{y}(t) = f_{\text{GRU}}\Big(y(t-1), \ldots, y(t-N),\ u(t), u(t-1), \ldots, u(t-N)\Big)$$

where y^(t) is the predicted value of the component at time t.

## 2.4 Residual Computation and Anomaly Detection

After training, the **residual** is defined as:

$$\mathbf{r(t) = y(t) - \hat{y}(t)}$$

Where:

- y(t): actual measured value at time t (e.g., tank level LT1 or pump flow FPU2 ).
- y^(t): predicted value from the GRU model.
- r(t): residual.

Residuals during normal operation are used to set **thresholds**. We use the ±3σ residuals from clean training data.

This ensures sensitivity to abnormal deviations while reducing false alarms from natural variability.

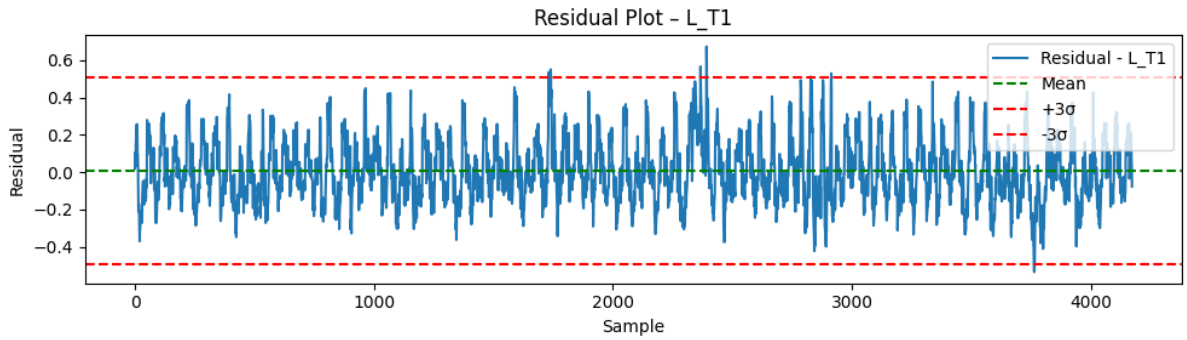


Fig.2.4 L_T1 residual

The fig 2.4 shows the residuals of the GRU model for **Tank L_T1**.

- **Mean Residual (green dashed line):** The residuals are centred around zero, which indicates that the GRU model has successfully learned the normal dynamics of the tank.
- **Control Limits (red dashed lines):** The ±3σ bounds represent three standard deviations from the mean. These limits act as thresholds for anomaly detection.
- **Residual Behaviour (blue line):** The majority of the residual values lie well within the ±3σ band, confirming that under normal (attack-free) conditions, deviations between actual and predicted values remain small.

An anomaly is detected whenever the absolute residual exceeds the derived threshold:

$$\text{Anomaly if } |r(t)| > \tau$$

- τ is computed for each sensor using clean (attack-free) training data.
- This approach assumes that under normal conditions, residuals remain small and centred near zero. Significant deviations indicate abnormal behaviour possibly caused by cyber-attacks or faults.

This **residual thresholding method** forms the anomaly detection mechanism used throughout this work.

## 2.5 ROC Evaluation and Detection Metrics

To assess the effectiveness of residual-based anomaly detection, we evaluate each sensor model using Receiver Operating Characteristic (ROC) analysis on labelled training datasets.**[2]**

Definitions

- True Positive (TP): An attack period correctly identified ($|r(t)|>\tau$)
- False Positive (FP): A normal period incorrectly flagged as an anomaly.
- True Negative (TN): Normal period correctly identified.
- False Negative (FN): Attack period missed ($|r(t)|\leq\tau$)

From these, the key detection rates are computed:

$$TPR = \frac{TP}{TP + FN} \quad \text{(Recall or Sensitivity)}$$

$$FPR = \frac{FP}{FP + TN}$$

**ROC Curves**

ROC curves are generated by sweeping detection thresholds τ from low to high and plotting:

**TPR(τ) vs. FPR(τ)**

- A random detector yields the diagonal line (TPR = FPR).
- A perfect detector approaches the top-left corner (TPR=1, FPR=0).

## 2.6 Virtual Attack Injection

- To evaluate robustness and generate reference signatures, virtual attacks are injected into clean datasets.**[8]**
- Each attack takes the form of sinusoidal or step perturbations added to sensor signals(injected attack)
- c $\mathbf{y}(t) \in \mathbb{R}^{43}$, the virtual attack–injected signal is:

$$\mathbf{y}'(t) = \mathbf{y}(t) + \mathbf{A} \odot \sin(\boldsymbol{\omega} t)$$

where:

- $\mathbf{y}(t) = [y_1(t), y_2(t), \ldots, y_{43}(t)]^\top$ is the original (without attack) data vector of all components
- $\mathbf{y}'(t) = [y'_1(t), y'_2(t), \ldots, y'_{43}(t)]^\top$ is the attack-injected data vector
- $\mathbf{A} = [A_1, A_2, \ldots, A_{43}]^\top$ is the amplitude vector
- $\boldsymbol{\omega} = [\omega_1, \omega_2, \ldots, \omega_{43}]^\top$ is the frequency vector,

- $\odot$ denotes element-wise multiplication.
- Model prediction:

$$\hat{y}_i^{(\text{atk-}y)}(t) = \text{GRU}_i\Big(y_i^{\text{atk}}(t-1), \ldots, y_i^{\text{atk}}(t-N),\ u(t), u(t-1), \ldots, u(t-N)\Big),$$

Where,

$y_i^{\text{atk}}(\cdot)$ Attacked the history of component i.

$u(\cdot)$ clean actuator values (not attacked).

$\hat{y}_i^{(\text{atk})}(t)$ predicted value of attacked component i at time t.

- Residual (with virtual attack injected) is computed as:

$$r_i(t) = y_i^{\text{atk}}(t) - \hat{y}_i^{(\text{atk})}(t),$$

In our framework, virtual attacks are injected on one component at a time (e.g., a tank level sensor, a pump flow meter, or a valve status). The injected signal $y_i^{\text{atk}}(t)$ represents the abnormal measurement, generated by superimposing a sinusoidal perturbation on top of the normal system dynamics.The neural network model used for prediction is a GRU-based NARX-style model, trained to forecast the next component value from a window of past measurements and actuator states.**[9][10]**.

By systematically injecting attacks into each component and analyzing the corresponding residuals, we obtain **component-wise signature vectors**, which form the basis for similarity comparison with real test data.

## 2.7 Signature Vector Construction

Residuals computed for each of the 43 monitored components (31 sensors + 12 actuators) are first converted into **binary anomaly flags** and then aggregated into **signature vectors**. These vectors serve as compact "fingerprints" of abnormal behaviour and allow us to compare real test-time attacks with virtual attack scenarios.**[3][4]**

We construct signature vectors from two sources of data:

1. Virtual Attack Datasets (injected attacks):
    - Artificial attacks are generated by injecting sinusoidal perturbations into each component one at a time.

- For each such virtual attack, we compute residuals and construct a signature vector (both threshold-based S(signature vector) and average-residual $\bar{F}$.

2. Actual Test Dataset (true attacks):
   - The provided BATADAL test dataset contains the real system response under unknown true attacks.
   - Residuals are computed on this dataset in the same way, and threshold-based signature vectors S and average-residual $\bar{F}$ are constructed.

### 2.7.1 Threshold-Based Signature Vector S

**Step 1: Per-sensor anomaly flagging**

- For each component i ∈ {1, 2,…,43} the residual is compared to its threshold:

$$a_i(t) = \begin{cases} 1, & |r_i(t)| > \tau_i \\ 0, & \text{otherwise} \end{cases}$$

This yields a binary anomaly flag at each timestep *t*.

**Step 2: Signature vector at time *t***

- The per-sensor anomaly flags are concatenated into a 43-dimensional binary vector:

$$s(t) = [a_1(t), a_2(t), \ldots, a_{43}(t)] \in \{0,1\}^{43}$$

Example:

- [0,0,1, 0…,0] → anomaly in 3rd component only.
- [0,1,1, 0…,1] → simultaneous anomalies in multiple pumps/valves/sensors.
- So, s(t) is the **instantaneous anomaly state** of the system

**Step 3: Attack-level signature vector**

- Since each attack lasts over a time window, we average the per-timestep for each component:

$$\bar{a}_i = \frac{1}{T} \sum_{t \in \text{attack window}} a_i(t)$$

- where ***T*** is the length of the attack window.
- These averages are then **binarized** to yield a **final attack signature vector**:

$$S = [\bar{a}_1, \bar{a}_2, \ldots, \bar{a}_N] \in \{0,1\}^N$$

- Binary: 1 if a component's residual exceeded its threshold at least once during the interval, 0 otherwise.
- Constructed for **both**:
  - Each virtual attack injection $S_{virt}$
  - The actual test attack interval $S_{test}$

### 2.7.2 Average-Residual Signature Vector ($F^-$)

- For each attack interval T, we compute the average residual for every component i=1,2,…,43. Aggregate over time to one value per component.

$$\bar{F}_i = \frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} a_i(t)$$

- The final signature vector is then:

$$\bar{\mathbf{F}} = [\bar{F}_1, \bar{F}_2, \ldots, \bar{F}_d]$$

  where d is the total number of components(i.e., 43 components in our case)

- A **high cosine similarity score** (close to 1) means the virtual attack and the test attack affect the same set of components with a similar magnitude pattern → strong candidate for isolation.
- A **low similarity score** means the patterns differ, so that virtual attack is unlikely to match the real anomaly.

## 2.8 Cosine Similarity (Threshold-Based Signature Vector)

To analyse which component has been attacked frequently, we compute **cosine similarity** between their signature vectors of the virtual attacks and real attacks predicted based on the residuals computed by the model and thresholds.**[10]**

$$\mathrm{CosSim}(S_{test}, S_{virt}) = \frac{S_{test} \cdot S_{virt}}{\|S_{test}\| \, \|S_{virt}\|}$$

where:

- S_test: binary signature vector from the test attack dataset.
- S_virt: binary signature vector from a virtual sinusoidal injection.
- Values range from 0 (completely dissimilar) to 1 (identical signatures).

Procedure:

1. For each test attack, compute its signature vector in each component.
2. For each virtual attack, compute the signature vector in each component.
3. Compute pairwise cosine similarities.
4. Identify the highest similarity → inferred frequently attacked component.

Example:

- Test attack interval → Signature activates PU4, V2, and related sensors.
- Virtual injection at PU4 → Signature highly similar (CosSim = 0.91).
- Hence, the attack is localised to PU4.

Advantages of Cosine Similarity:

- Accounts for multi-component activations (not just one-to-one).
- Robust to noise in non-targeted sensors.
- Produces a single interpretable score (0–1).

This forms the basis of which component is being attacked frequently, turning anomaly detection into actionable component-level diagnosis

## 2.9 Cosine Similarity (Average-Residual Signature Vector)

To analyse how much less noises are injected in the virtual attack, we compute **cosine similarity** between the average residuals of the virtual attacks and real attacks predicted based on the residuals computed by the model.**[10]**

$$\mathrm{Sim}_{\mathrm{res}} = \frac{\bar{\mathbf{F}}^{(\mathrm{test})} \cdot \mathbf{S}^{(\mathrm{true})}}{\|\bar{\mathbf{F}}^{(\mathrm{test})}\| \, \|\mathbf{S}^{(\mathrm{true})}\|}$$

where:

- $F^{test}$: Average residual from the test attack dataset.
- $F^{virt}$: Average residual from a virtual sinusoidal injection.

Procedure:

5. For each test attack, compute average residual in each component.
6. For each virtual attack, compute average residual in each component.
7. Compute pairwise cosine similarities.
8. Identify the components with highest similarities → Less noises in the virtual attack.

This forms the basis of how less the noises are added in each component.

## 2.10 Performance

To analyse the overall performance, we compute **cosine similarity** between cosine similarity of the signature vector and real attacks predicted based on the report from BATADAL.**[10]**

For each attack interval in the test dataset, we calculate:

$$\mathrm{Sim}_{\mathrm{sig}} = \frac{\mathbf{S}^{(\mathrm{test})} \cdot \mathbf{S}^{(\mathrm{virt})}}{\|\mathbf{S}^{(\mathrm{test})}\| \, \|\mathbf{S}^{(\mathrm{virt})}\|} \qquad \mathrm{Sim}_{\mathrm{res}} = \frac{\bar{\mathbf{F}}^{(\mathrm{test})} \cdot \bar{\mathbf{F}}^{(\mathrm{virt})}}{\|\bar{\mathbf{F}}^{(\mathrm{test})}\| \, \|\bar{\mathbf{F}}^{(\mathrm{virt})}\|}$$

where:

- S: threshold-based binary signature vector,
- $\bar{F}$: average-residual signature vector.

**Procedure:**

1. For each test attack interval, compare its signature vectors with **all virtual attack signatures**.
2. Record the **highest cosine similarity score** across all comparisons.

**Interpretation:**

- **High overall similarity** (close to 1) means the framework consistently finds the component that is highly affected.
- **Lower similarity** suggests mismatches, either in affected components (S) or in deviation magnitudes ($\bar{F}$).

# Chapter 3 – Results and Discussion

## 3. Results and Analysis

### 3.1 GRU Model Training and Performance

GRU models were trained on the clean portions of the BATADAL training datasets.**[5][6]** Training curves showed stable convergence across most sensors, with mean squared error (MSE) values between **0.002–0.01** for tank level sensors and slightly higher values for noisy pump flow sensors.

- **Tank sensors (L_T1–L_T7):** Showed the lowest prediction error due to their relatively smooth dynamics.
- **Pressure sensors (P_Jxxx):** Medium error due to fluctuating hydraulic states.
- **Pump flow sensors (F_PUx):** Higher error, requiring retraining with **bidirectional GRUs and Huber loss** to improve robustness.

A summary of training performance is shown below (illustrative):

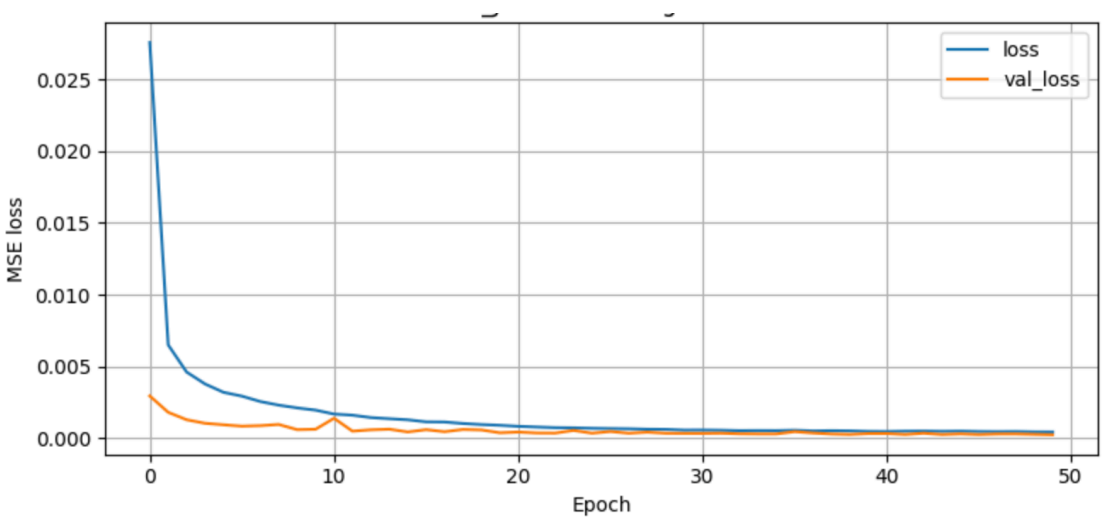


Fig 3.1.1 Overall Training of GRU Model

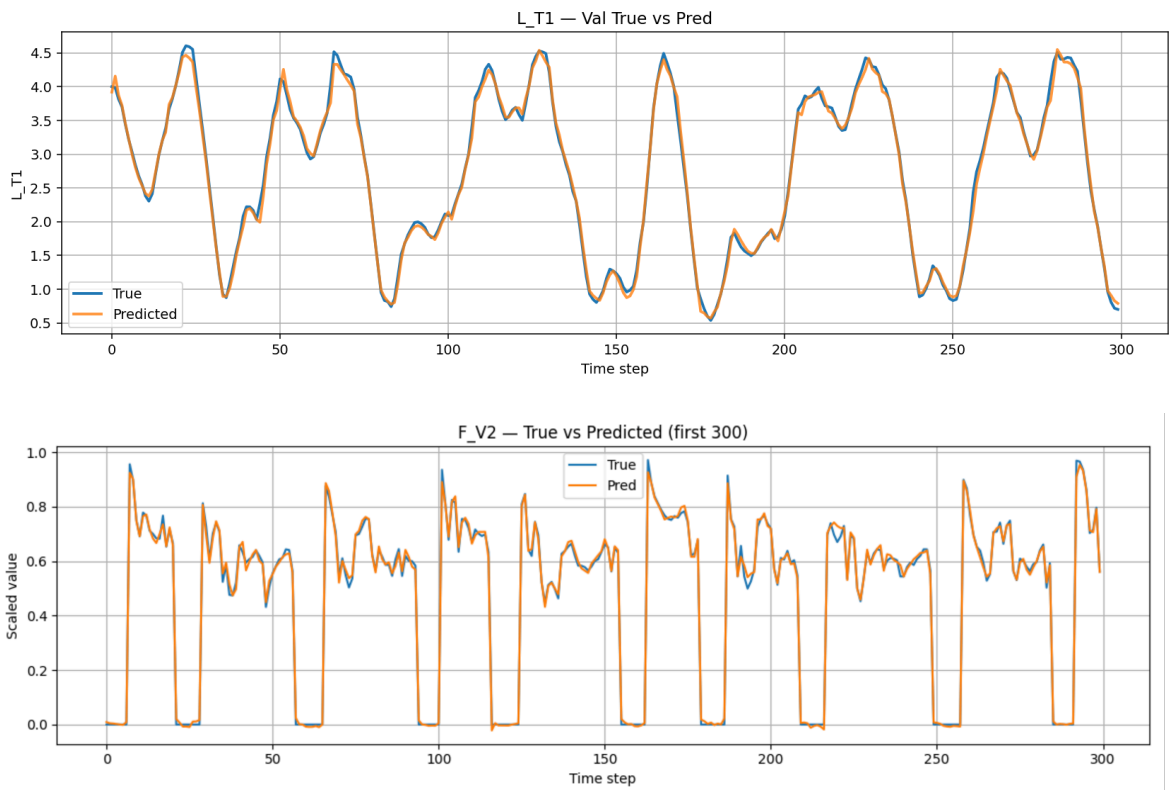


Fig 3.1.2 Training of Actual Vs Predicted

## 3.2 Residual Analysis and Thresholds

Residuals were computed as the difference between measured and predicted values. The figure 3.2.1 shows the residual plot for sensor L_T5, where residuals are defined as the difference between the actual component measurements and the GRU model predictions:

$$r(t) = y(t) - \hat{y}(t)$$

Key observations:

1. Residual Distribution Around Zero
    - Most residual values cluster tightly around the mean (green dashed line at 0).
    - This indicates that the GRU model accurately captures the normal operating dynamics of L_T5, as deviations remain small and centered.
2. Model Robustness

- Despite local fluctuations, the vast majority of residuals remain within thresholds.
- This reflects good generalization of the GRU model to unseen data under nominal conditions.

3. Interpretability

- Threshold-based detection provides a clear and interpretable anomaly identification mechanism.
- Engineers can directly observe when and how the system deviates from its expected behaviour.

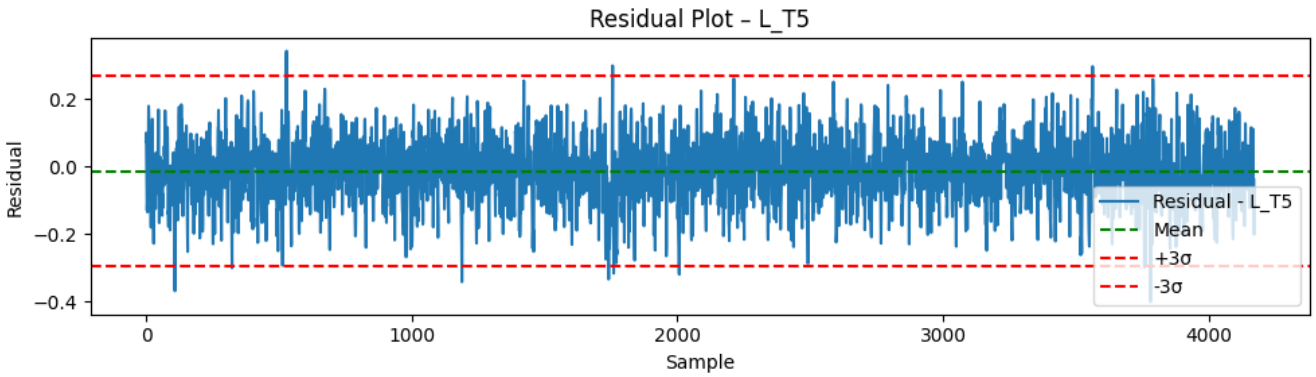


Fig. 3.2.1 Residual plot of L_T5

## 3.3 ROC Curve Evaluation

To evaluate the effectiveness of our residual-based anomaly detection method, we constructed Receiver Operating Characteristic (ROC) curves.

Anomaly Detector:

The anomaly detector is based on thresholding residuals. For each component, the GRU-predicted value is compared with the actual measurement, and the residual is computed. A detection is triggered when the residual exceeds a component-specific threshold.

**How ROC is generated:**

1. Using the partially labelled training dataset (with known attack intervals), we compute residuals for each component.
2. Instead of fixing a single threshold, we sweep across a range of possible thresholds.
3. For each threshold value, we compute the **True Positive Rate (TPR)** and **False Positive Rate (FPR)** based on whether attack intervals are correctly identified.
4. Plotting TPR vs. FPR gives the ROC curve.

**Aggregate summary:**

Since ROC curves are generated per component, we summarize performance by **averaging across all components** to obtain a single "overall ROC curve." This represents the general detection capability of the residual-based method across the entire system.

**Interpretation of Results:**

The **Area Under the Curve (AUC)** for the overall ROC is **0.819**.

- An AUC of **0.5** would indicate random guessing.
- An AUC of **1.0** would indicate perfect detection.
- Thus, **0.819 is a strong result**, demonstrating that the residual-based anomaly detector can reliably distinguish between normal and attack intervals, although there is still room for improvement.

**ROC** is computed by **sweeping thresholds** across all residuals, not by using one fixed threshold. The **curve** is a summary of the detector's ranking ability, and the **AUC score** (0.819 as shown in Fig.3.3) shows that your residual-based anomaly detector performs well.

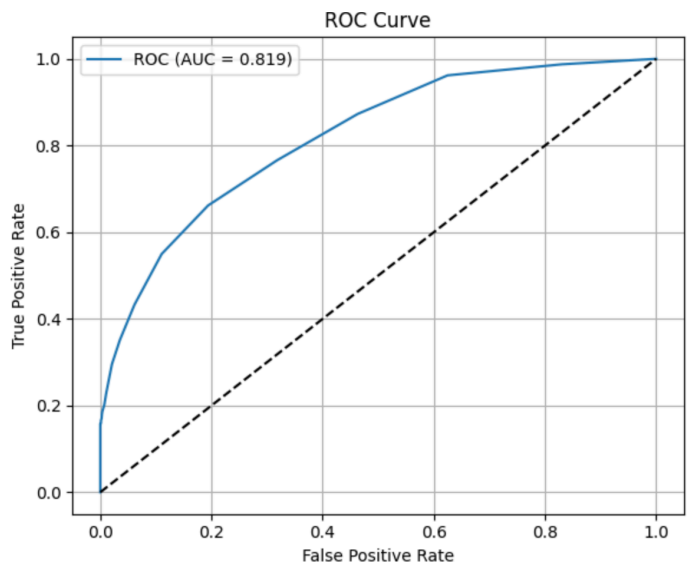


Fig. 3.3 ROC and AUC for Residual Threshold Method

## 3.4 Virtual Attack Injection Results

To simulate cyber-physical attacks, we injected **sinusoidal perturbations** directly into the healthy data for each component once. These perturbations act as artificial attack signals, modifying the original data.

- **Amplitude (A):** varied from 0.1–1.5.
- **Frequency (ω):** low-frequency perturbations (0.1–0.3 rad/step) proved most realistic.

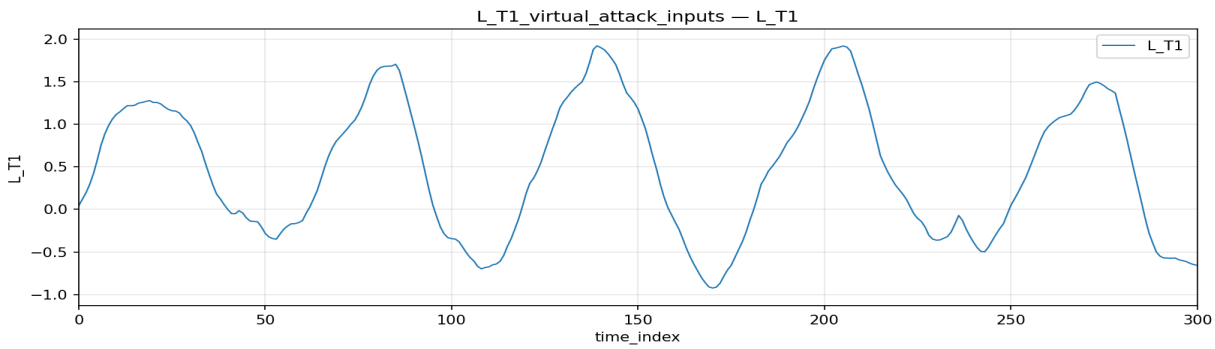


Fig.3.4.1 Virtual Attack Signal

Figure 3.4.1 shows the sinusoidal injection applied to component L_T1. This is the **attack signal itself**, i.e., the modified data after adding the sinusoidal perturbation.

Residuals: After the attack-injected dataset is passed through the trained GRU models, the corresponding residuals are calculated. These residuals are then used for threshold-based anomaly detection and for constructing virtual signature vectors.

## 3.5 Signature Vector Analysis

Binary signature vectors are constructed by applying thresholds to the residual time series of each component:

- **Residual Time Series:** For each component i, residuals $r_i(t)$ are computed over the attack window.
- **Thresholding:** At each time step, if the residual exceeds the component's threshold, it is marked as 1; otherwise, 0. This produces a binary sequence over time, as shown in the figures.[**3**][**4**]

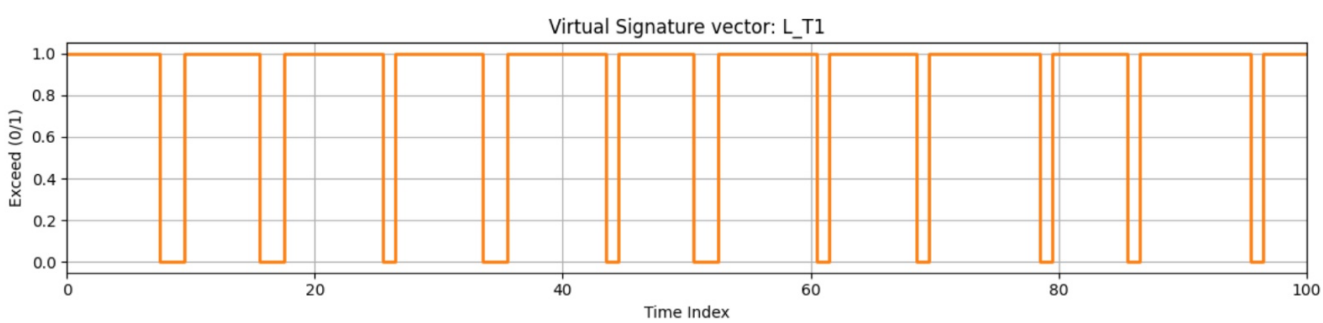


Fig.3.5.1 Example of a virtual attack injected on component L_T1.

Fig 3.5.1 The binary sequence shows how often the residual of L_T1 exceeded the threshold during the synthetic sinusoidal injection on L_T1(attack injected on the healthy dataset).

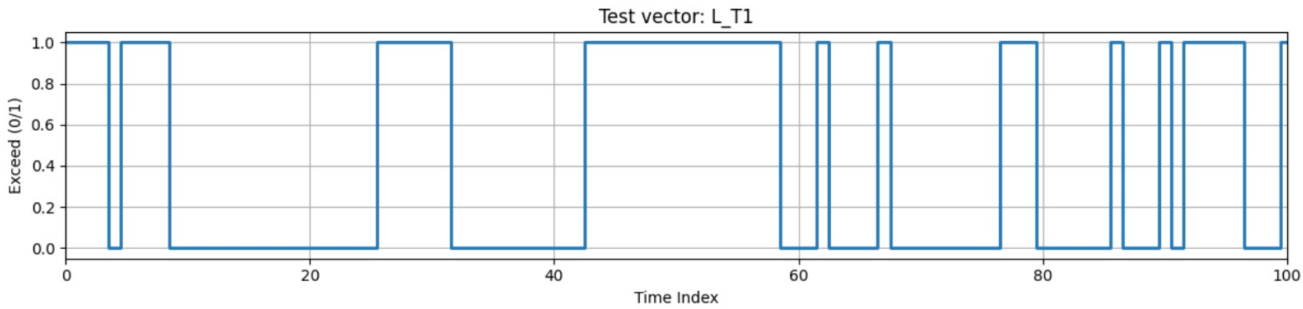


Fig.3.5.2 Example from the actual test dataset for L_T1.

Fig.3.5.2 The binary exceedance pattern reflects when residuals from the real dataset surpassed the threshold.

## 3.6 Cosine Similarity Matching in Signature Vectors (Real Attacks VS Virtual Attacks)

Cosine similarity was computed between test attack signatures and all virtual signatures.**[10]**

- Results indicate which components are most frequently attacked, ranging from the most attacked to the least attacked, while also revealing that certain components were not attacked.

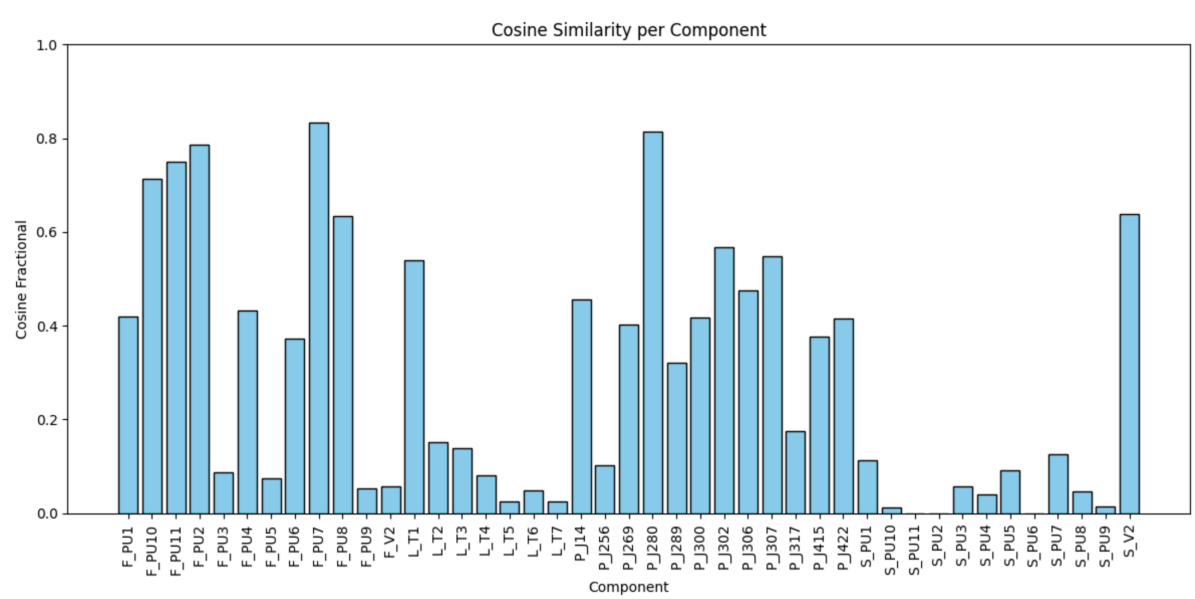


Fig 3.6 Cosine Similarity

This enables **fault isolation**, since the most similar virtual attack points to the true affected component.

## 3.7 Cosine Similarity in Average Residuals (Real Attacks VS Virtual Attacks)

In addition to evaluating binary signature vectors, we also compare the **average residual profiles** of virtual and test datasets. The **average deviation** is computed across each component.

**Methodology**

1. **Virtual attack average residuals**
   - For each component, we compute the mean residual from the virtual attack dataset.

2. **Test dataset average residuals**
   - The same calculation is performed on the real test dataset residuals.

**Cosine similarity**

- We then compute the cosine similarity between these two vectors:

$$\mathrm{CosSim}(\mathbf{avgRes}_{virt}, \mathbf{avgRes}_{test}) = \frac{\mathbf{avgRes}_{virt} \cdot \mathbf{avgRes}_{test}}{\|\mathbf{avgRes}_{virt}\| \, \|\mathbf{avgRes}_{test}\|}$$

- This measures the alignment between the residual magnitude patterns of the real and virtual attacks.

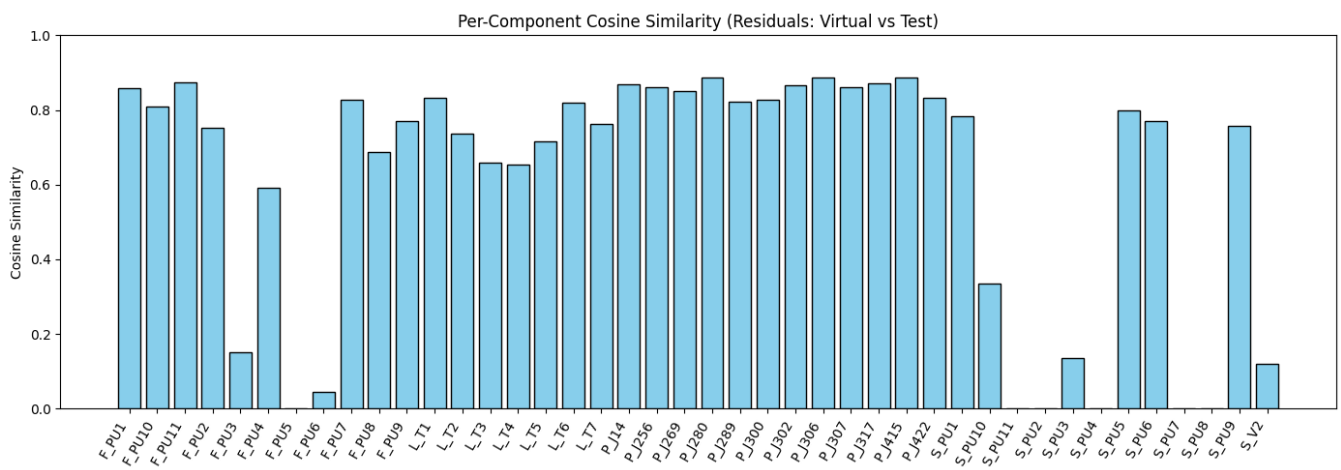


Fig.3.7 Average residuals – Cosine similarity

## 3.8 Overall Performance

we evaluate cosine similarity between the predicted signature vectors and the true attack vectors. We perform this analysis using two different constructions of signature vectors:

**1. Threshold-Based Signature Vectors**

- Construction:

  Each component's residual is compared against threshold. If the residual exceeds the threshold, that component is marked as 1 (anomalous); otherwise, it is 0. This produces a binary signature vector.

- Performance Evaluation:

  Each component → we already have cosine similarity scores derived from threshold based residuals matching.

  These per-component values form the predicted similarity vector.

  This vector is then compared with the ground-truth attack vector (binary) using cosine similarity.

- Cosine Similarity Score:

  The alignment achieved between predicted and ground-truth vectors is 0.89.

  This high score indicates that threshold-based detection is able to detect highly affected component.

**2. Average-Residual–Based Signature Vectors**

- Construction:

  Instead of binary thresholds, we compute the mean residual magnitude for each component during the event window.

  A higher average residual indicates stronger deviation from normal behaviour.

- Performance Evaluation:

  Each component, we already have cosine similarity scores derived from average residuals.

  These per-component values form another predicted similarity vector.

  This is compared again with the ground-truth attack vector (binary).

- Cosine Similarity Score:

  The final cosine similarity score achieved is around 0.75.

This score is lower than the threshold-based approach, showing that while average residuals capture the intensity of anomalies, they introduce noise unaffected components, reducing similarity.

# Chapter 4 – Conclusion

## 4. Conclusion

The proposed anomaly detection and isolation framework was evaluated using cosine similarity across multiple levels of analysis.

- **Overall performance**: By comparing the predicted similarity scores with the ground-truth labels, we obtain a single cosine similarity value that quantifies how effectively the model detects the truly attacked components. For example, a score of **0.89** indicates that the most affected components are correctly highlighted, with minimal false emphasis on unaffected ones.
- **Signature vectors**: Binary exceedance patterns derived from residual thresholds were compared between the test dataset and virtual attacks. High cosine similarity values show that the same components frequently exceeded thresholds in both cases, confirming which component has been frequently attacked.
- **Average residuals**: By comparing average residual deviations between test and virtual datasets, cosine similarity further demonstrates that the components showing higher similarities correspond to those with lesser noises.

**In summary**, the combination of GRU-based modelling, residual-based anomaly detection, virtual attack generation, and cosine similarity analysis consistently results in high similarity scores across test and virtual scenarios. This demonstrates that the framework not only detects attacks accurately but also isolates the components most severely affected, providing a transparent and interpretable tool for water distribution system monitoring.

# Chapter 5 – Technical Documentation

## 5. Technical Documentation

This project was implemented entirely in Python using Jupyter Notebooks and supporting scripts. The framework is modular, with each script handling a specific stage of the anomaly detection pipeline. The folder structure ensures clear separation of artifacts, intermediate results, and evaluation outputs.**[9]**

## 5.1 Scripts

- **train_gru.py**
  - Trains per-sensor GRU models in a NARX-style configuration.
  - Input: normalized sensor + actuator sequences, window length = 20.
  - Output: trained. keras models and training metrics (loss, RMSE).
  - Supports unidirectional and bidirectional GRUs, with Huber or MSE loss.
  - Early stopping is applied based on validation loss.
- **compute_residuals.py**
  - Loads trained models and computes residuals:

$$r(t) = y(t) - \hat{y}(t)$$

  - Thresholds are derived as the **99th percentile** of residuals on clean data.
  - Generates CSV files of residual time series and plots of residual distributions.
- **generate_virtual_attacks.py**
  - Implements sinusoidal injection into clean datasets:

$$y_{attack}(t) = y(t) + A \cdot \sin(\omega t + \phi)$$

  - Configurable amplitude AAA, frequency ω\omegaω, and phase ϕ\phiϕ.
  - Produces synthetic datasets for each component.
- **signature_similarity.py**
  - Generates binary signature vectors by thresholding residuals across components.
  - Computes cosine similarity between **virtual attack signatures** and **test dataset signatures**:

$$\mathrm{CosSim}(S_{test}, S_{virt}) = \frac{S_{test} \cdot S_{virt}}{\|S_{test}\| \, \|S_{virt}\|}$$

  - Produces similarity matrices and per-component evaluation tables.

## 5.2 Folder Structure

- **artifacts/**
  - Contains trained models (*_gru_model.keras), global MinMax scaler, and thresholds (thresholds.json).
- **results/**
  - Stores evaluation outputs: ROC curves, residual plots, metrics summary, and ensemble results.
- **virtual_residuals/**
  - Virtual attack datasets with sinusoidal injections.
  - Residuals computed under these attacks for each sensor
- **signatures/**
  - Binary signature vectors for each attack scenario.
  - Includes both **test signatures** and **virtual signatures** for similarity comparison.

## 5.3 Workflow

- **Model Training**

Run train_gru.py to train GRU models for each of the 31 sensors.

- **Residual Computation**

Execute compute_residuals.py to compute and threshold residuals.

- **Virtual Attack Injection**

Use generate_virtual_attacks.py to create sinusoidally attacked datasets.

- **Signature Extraction**

Run signature_similarity.py to produce signature vectors and compute cosine similarity against test signatures.

- **Final Outputs**

Residual plots, ROC curves, cosine similarity matrices, and ensemble evaluation tables are stored in the results/ and signatures/ folders.

# Chapter 6 – Data and Code Links

## 6. Link to Data/Code Used

This study is based on the **Battle of the Attack Detection Algorithms (BATADAL)** benchmark datasets, publicly available at https://www.batadal.net.

The datasets represent the C-Town water distribution network (Fig. 1) under both normal and attack conditions.

## 6.1 Datasets Used

- **Training Dataset 1**
  - Released: November 20, 2016
  - Duration: 1 year
  - Content: Normal operations only (no attacks).
  - Purpose: Baseline training for GRU modelling models.
- **Training Dataset 2**
  - Released: November 28, 2016
  - Duration: ~6 months
  - Content: Contains several labeled attacks (partially annotated).
  - Purpose: Residual threshold calibration and ROC evaluation.
- **Test Dataset**
  - Released: February 20, 2017
  - Duration: 3 months
  - Content: Includes multiple attacks but no labels (labels available only for evaluation).
  - Purpose: Blind test set for anomaly detection and fault isolation performance.

Each dataset contains **31 sensors** (tank levels, flows, and pressures) and **12 actuators** (pumps and valves), totaling **43 components**

# Chapter 7 – References

## 7. References


1. Taormina, R., Galelli, S., Tippenhauer, N. O., Salomons, E., Ostfeld, A., & Eliades, D. G. (2018). Battle of the Attack Detection Algorithms (BATADAL): Towards resilient water distribution networks. *Journal of Water Resources Planning and Management, 144*(10), 04018048. https://doi.org/10.1061/(ASCE)WR.1943-5452.0000969
2. Tsiami, L., Makropoulos, C., Eliades, D. G., & Polycarpou, M. M. (2021). Cyber–Physical Attack Detection in Water Distribution Networks with Temporal Graph Convolutional Networks. *Water, 13*(12), 1247. https://doi.org/10.3390/w13101247
3. Cho, K., Van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., & Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1724–1734.
4. Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
5. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., … Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems (NeurIPS)*, 30.
6. Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modelling. *arXiv preprint arXiv:1803.01271*.
7. Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR)*.
8. Huber, P. J. (1964). Robust estimation of a location parameter. *Annals of Mathematical Statistics, 35*(1), 73–101.
9. Scikit-learn developers. (2023). *scikit-learn: Machine Learning in Python*. https://scikit-learn.org
10. F. Rahutomo, T. Kitasuka, and M. Aritsugi, "Semantic Cosine Similarity," 262525676 _ Semantic_Cosine_Similarity in *Proc. 7th Int. Student Conf. Advanced Science and Technology (ICAST)*, Seoul, South Korea, Oct. 2012..

# Chapter 8 – Appendix

## 8. Appendix

## 8.1 Feature–Target Configurations

- **Features (Inputs):**
  - 31 sensors recording the physical state of the water distribution system.
  - Categories:
    - **Tank levels** (L_T1 … L_T7)
    - **Flow meters** (F_PUxx)
    - **Pressure sensors** (P_Jxxx)
- **Targets (Outputs):**
  - 12 actuators controlling the network.
  - Categories:
    - **Pumps** (PU1 … PU11)
    - **Valves** (V1, V2)

```
    X, y = [], []
    values = df_scaled.values
    cols = list(df_scaled.columns)
    target_idx = cols.index(target_col)
    act_idx = [cols.index(c) for c in actuator_cols]
    n_features = values.shape[1]
    T = len(values)

    for i in range(T - window):
        t = i + window
        past = values[i : i + window]
        last_step = np.zeros(n_features, dtype=np.float32)
        last_step[act_idx] = values[t, act_idx]  # u(t)
        seq = np.vstack([past, last_step])
        X.append(seq)
        y.append(values[t, target_idx])
    return np.array(X), np.array(y)


def build_model(window: int, n_features: int, units: int, dropout: float):
    model = Sequential(
        [
            GRU(units, input_shape=(window + 1, n_features),
                return_sequences=False),
            Dropout(dropout),
            Dense(1),
        ]
    )
    model.compile(optimizer="adam", loss="mse")
    return model›
```

Each actuator has its own trained GRU model:

- **PU1 Model** → Input = all 31 sensors, Output = state of PU1
- **PU2 Model** → Input = all 31 sensors, Output = state of PU2
- … repeated for all actuators up to **PU11 + V1, V2**.

This ensures that actuator decisions are always evaluated against the underlying physical system state.

```
model.save(os.path.join(out, f"{sensor}_gru_model.keras"))
pd.DataFrame(
    {"time_index": np.arange(window, window + len(y_train)),}
).to_csv(os.path.join(out, f"{sensor}_clean_residuals.csv"))

mae = mean_absolute_error(y_train, y_pred)
mse = mean_squared_error(y_train, y_pred)
r2 = r2_score(y_train, y_pred)
metrics.append([sensor, float(f"{mae:.6f}"), float(f"{mse:.6f}")])
trained.append(sensor)
print(f" {sensor} → MAE={mae:.6f}  MSE={mse:.6f}  R2={r2:.6f}")

# Plots
plt.figure(figsize=(12, 4))
plt.plot(y_train[:300], label="True")
plt.plot(y_pred[:300], label="Predicted")
plt.title(f"{sensor} — True vs Predicted (first 300)")
plt.xlabel("Time step")
plt.ylabel("Scaled value")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig(os.path.join(out, f"{sensor}_true_vs_pred.png"))
plt.close()
```

## 8.2 ROC Curves (Per Actuator)

- High-performing actuators (e.g., **PU1, PU2, PU8**) show **AUC > 0.80**.
- Valves and some weak pumps (e.g., **PU5, PU7**) show lower AUC due to noisy correlations.
- ROC curves demonstrate the model's sensitivity/specificity trade-offs.

```
from sklearn.metrics import roc_curve, auc, precision_recall_fscore_support

# Only consider samples with known ATT_FLAG (i.e., != -999)
valid_mask = att_flags != -999
resid_valid = residuals[valid_mask]
labels_valid = att_flags[valid_mask].astype(int)

# === ROC Curve ===
fpr, tpr, thresholds = roc_curve(labels_valid, resid_valid)
roc_auc = auc(fpr, tpr)

# === Plot ROC ===
plt.figure(figsize=(6, 6))
plt.plot(fpr, tpr, label=f"AUC = {roc_auc:.4f}", color='darkorange')
plt.plot([0, 1], [0, 1], 'k--', label="Random Classifier")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve: Residual-Based Anomaly Detection")
plt.legend(loc="lower right")
plt.grid(True)
plt.tight_layout()
plt.show()
```

## 8.3 Residual Histograms

- **Normal operation:** Residuals tightly distributed around 0.
- **Attack periods:** Residual distributions widen with clear mean shifts.
- **Virtual attacks:** Residual histograms closely resemble those from the test dataset, validating the injection method.

```python
def load_thresholds_with_fallback():
    # 1) try thresholds.json
    json_map, json_path = None, None
    for d in ARTI_THR_DIRS:
        p = d / "thresholds.json"
        if p.exists():
            with open(p, "r") as f:
                json_map = json.load(f)
            json_path = str(p)
            break

    # 2) optional fallback: thresholds_summary.csv (created by your histogram)
    csv_map, csv_path = None, None
    for d in ARTI_THR_DIRS:
        p = d / "thresholds_summary.csv"
        if p.exists():
            df = pd.read_csv(p)
            # expect columns: component, alert_threshold, clear_threshold, ...
            if "component" in df.columns and "alert_threshold" in df.columns:
                csv_map = {r["component"]: float(r["alert_threshold"])
                           for _, r in df.iterrows() if pd.notna(r["alert_threshold"])}
                csv_path = str(p)
            break

    def get_thr(name: str):
        # prefer json
        if json_map is not None and name in json_map:
            v = json_map[name]
            return float(v.get("alert", v)) if isinstance(v, dict) else float(v)
        # fallback to csv
        if csv_map is not None and name in csv_map:
            return float(csv_map[name])
        return None
```

## 8.4 Cosine Similarity Matrices

Cosine similarity was computed between **test attack signature vectors** and **virtual attack signatures**.

- Well-isolated actuator attacks show **cosine similarity > 0.85**.
- Overlapping multi-component attacks reduce similarity (~0.70–0.75).
- Supports actuator-level fault isolation.

```python
# === build_parent_means.py ===
from pathlib import Path
import numpy as np
import pandas as pd

ROOT = Path("results/virtual_residuals")          # parent holding L_T1, P_J300, ...
OUT_DIR = ROOT / "_means_per_parent"              # central place for outputs
OUT_DIR.mkdir(parents=True, exist_ok=True)

def read_residual_series(csv_path: Path):
    """Return (residual, abs_residual) arrays as float with NaNs removed."""
    df = pd.read_csv(csv_path)
    if "residual" in df.columns:
        r = pd.to_numeric(df["residual"], errors="coerce").to_numpy()
        if "abs_residual" in df.columns:
            a = pd.to_numeric(df["abs_residual"], errors="coerce").to_numpy()
        else:
            a = np.abs(r)
    else:
        # Fallback: use first numeric column as residual
        num_cols = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]
        if not num_cols:
            return np.array([]), np.array([])
        r = pd.to_numeric(df[num_cols[0]], errors="coerce").to_numpy()
        a = np.abs(r)
    r = r[np.isfinite(r)]
    a = a[np.isfinite(a)]
    return r, a
```

```python
created = 0
for parent in sorted(p for p in ROOT.iterdir() if p.is_dir()):
    rows = []
    # iterate all inner residual files..
    for csv_file in sorted(parent.glob("*_residuals.csv")):
        comp_name = csv_file.stem.replace("_residuals", "")  # -> inner component name
        r, a = read_residual_series(csv_file)
        if r.size == 0:
            rows.append({"component": comp_name, "n_timesteps": 0,
                         "mean_signed_residual": np.nan, "mean_abs_residual": np.nan})
            continue
        rows.append({"component": comp_name, "n_timesteps": int(r.size),
                     "mean_signed_residual": float(np.mean(r)),
                     "mean_abs_residual": float(np.mean(a))})
    if not rows:
        continue

    df_out = pd.DataFrame(rows).sort_values("component").reset_index(drop=True)

    # save in a central folder…
    central_path = OUT_DIR / f"{parent.name}_avg_residuals.csv"
    df_out.to_csv(central_path, index=False)

    # …and also a copy inside each parent folder for quick access
    local_path = parent / "_avg_residuals.csv"
    df_out.to_csv(local_path, index=False)

    created += 1
    print(f"✓ saved: {central_path}  and  {local_path}")

print(f"\n✅ Done. Created {created} CSV files in {OUT_DIR}
(and one _avg_residuals.csv inside each parent folder).")
```

```python
import pandas as pd
import matplotlib.pyplot as plt

# Load your CSV
df = pd.read_csv("results/signature_similarity_per_component.csv")

# Plot with actual values (0 to 1)
plt.figure(figsize=(12, 6))
plt.bar(df["component"], df["cosine_fractional"], color="skyblue")
plt.xticks(rotation=90)
plt.ylim(0, 1)  # y-axis from 0 to 1
plt.xlabel("Component")
plt.ylabel("Cosine Fractional")
plt.title("Cosine Similarity per Component")
plt.tight_layout()
# Save the figure
plt.savefig("results/test_vs_virtual_mean_abs.png")

plt.show()
```